

Module wise backend dev guidelines for LLM

Admitix Backend Module Implementation Guide

System Roles Definition

Overview

The Admitix backend follows a **Role-Based Access Control (RBAC)** architecture. Every authenticated user is assigned a role, and each role determines the resources and operations that the user can access throughout the system.

System Roles

Role	Description	Access Level
Super Admin	Manages the complete Admitix platform including institutions, users, roles, system configuration, and monitoring.	Full System Access
College Admin	Manages admissions, departments, courses, students, documents, staff, and institution-level settings.	Institution Level
Admission Officer	Reviews applications, verifies documents, manages admission workflow, and processes student admissions.	Admission Operations
Student	Accesses personal profile, submits applications, uploads documents, tracks admission status, and views notifications.	Self Access

Standard Permission Types

Permission	Description
Create	Create new records
Read	View records
Update	Modify existing records
Delete	Soft delete records
Manage	Administrative operations

Module 1 – Roles

Module Information

Property	Value
Module Name	Roles
Depends On	None
Used By	Users, Auth, Students, Applications, Documents, Payments, Notifications, Audit Logs
Primary Table	<code>roles</code>

1. Module Overview

Purpose

The **Roles** module manages the roles available in the Admitix system. It acts as the foundation of the Role-Based Access Control (RBAC) mechanism by defining roles that determine user permissions across all modules.

Responsibilities

- Create new system roles.
- Retrieve role information.
- Update existing roles.
- Activate or deactivate roles.
- Prevent duplicate role creation.
- Provide role information to dependent modules.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/roles</code>	<code>create_role()</code>	Create a new role
GET	<code>/roles</code>	<code>get_roles()</code>	Retrieve all roles

Method	Endpoint	Service Function	Description
GET	/roles/{id}	get_role()	Retrieve role details
PUT	/roles/{id}	update_role()	Update role information
DELETE	/roles/{id}	delete_role()	Delete/archive a role

(Endpoints are based on your API specification.)

3. Service Implementation

create_role()

Item	Details
Purpose	Create a new role in the system.
Repository	get_role_by_name() , create_role()
Validation	Role name is required and must be unique.
Permission	Super Admin
Exceptions	RoleAlreadyExistsException , ValidationException
Logic	Validate request → Check permission → Verify duplicate role → Save role → Return success response

get_roles()

Item	Details
Purpose	Retrieve all available roles.
Repository	get_roles()
Validation	Validate pagination, filters, and sorting parameters.
Permission	Super Admin, College Admin
Exceptions	ValidationException
Logic	Validate request → Retrieve roles → Apply filters & pagination → Return response

get_role()

Item	Details
Purpose	Retrieve details of a specific role.

Item	Details
Repository	<code>get_role_by_id()</code>
Validation	Validate Role UUID.
Permission	Super Admin, College Admin
Exceptions	<code>RoleNotFoundException</code>
Logic	Validate ID → Retrieve role → Return response

update_role()

Item	Details
Purpose	Update role information.
Repository	<code>get_role_by_id()</code> , <code>get_role_by_name()</code> , <code>update_role()</code>
Validation	Role must exist and updated role name must remain unique.
Permission	Super Admin
Exceptions	<code>RoleNotFoundException</code> , <code>RoleAlreadyExistsException</code>
Logic	Validate request → Retrieve role → Check duplicate role name → Update role → Return success response

delete_role()

Item	Details
Purpose	Soft delete an existing role.
Repository	<code>get_role_by_id()</code> , <code>check_role_usage()</code> , <code>delete_role()</code>
Validation	Role must exist and should not be assigned to active users.
Permission	Super Admin
Exceptions	<code>RoleNotFoundException</code> , <code>RoleInUseException</code>
Logic	Validate ID → Retrieve role → Verify role usage → Soft delete role → Return success response

4. Module Notes

Database Tables

- `roles`

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User

Implementation Notes

- Use UUID as the primary key.
- Implement soft delete using the `is_deleted` flag.
- Prevent duplicate role names.
- Do not delete roles assigned to active users.
- Keep business logic inside `service.py`.
- Restrict `repository.py` to database operations only.
- Return standardized API responses using the common response utility.
- Record all create, update, and delete operations in the Audit Logs module.

Module 2 – Users

Module Information

Property	Value
Module Name	Users
Depends On	Roles, Institutions
Used By	Auth, Students, Staff, Notifications, Chat, Audit Logs
Primary Table	<code>users</code>

1. Module Overview

Purpose

The **Users** module manages all user accounts in the Admitix system. It stores user information, associates users with roles and institutions, and provides user data for authentication, authorization, and other dependent modules.

Responsibilities

- Create and manage user accounts.
- Retrieve user information.
- Update user details.
- Activate or deactivate users.
- Reset user passwords.
- Provide user information to dependent modules.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	/users	create_user()	Create a new user
GET	/users	get_users()	Retrieve all users
GET	/users/{id}	get_user()	Retrieve user details
PUT	/users/{id}	update_user()	Update user information
PATCH	/users/{id}/status	update_user_status()	Activate/Deactivate user
PATCH	/users/{id}/reset-password	reset_password()	Reset user password
DELETE	/users/{id}	delete_user()	Soft delete user

(Endpoints are based on your API specification.)

3. Service Implementation

create_user()

Item	Details
Purpose	Create a new user account.

Item	Details
Repository	<code>get_user_by_email()</code> , <code>get_role_by_id()</code> , <code>get_institution_by_id()</code> , <code>create_user()</code>
Validation	Name, email, mobile, role and institution are required. Email and mobile must be unique.
Permission	Super Admin, College Admin
Exceptions	<code>UserAlreadyExistsException</code> , <code>RoleNotFoundException</code> , <code>InstitutionNotFoundException</code> , <code>ValidationException</code>
Logic	Validate request → Check permission → Verify role → Verify institution → Check duplicate email/mobile → Hash password → Create user → Return success response

get_users()

Item	Details
Purpose	Retrieve all users.
Repository	<code>get_users()</code>
Validation	Validate pagination, filters and sorting parameters.
Permission	Super Admin, College Admin
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve users → Apply filters & pagination → Return response

get_user()

Item	Details
Purpose	Retrieve user details.
Repository	<code>get_user_by_id()</code>
Validation	Validate User UUID.
Permission	Super Admin, College Admin, Self
Exceptions	<code>UserNotFoundException</code>
Logic	Validate ID → Retrieve user → Return response

update_user()

Item	Details
Purpose	Update user information.
Repository	<code>get_user_by_id()</code> , <code>get_user_by_email()</code> , <code>update_user()</code>
Validation	User exists. Updated email and mobile must remain unique.
Permission	Super Admin, College Admin
Exceptions	<code>UserNotFoundException</code> , <code>UserAlreadyExistsException</code>
Logic	Validate request → Retrieve user → Check duplicate email/mobile → Update user → Return success response

update_user_status()

Item	Details
Purpose	Activate or deactivate a user account.
Repository	<code>get_user_by_id()</code> , <code>update_user_status()</code>
Validation	User exists. Status value is valid.
Permission	Super Admin, College Admin
Exceptions	<code>UserNotFoundException</code> , <code>InvalidStateException</code>
Logic	Validate ID → Retrieve user → Update status → Return success response

reset_password()

Item	Details
Purpose	Reset a user's password.
Repository	<code>get_user_by_id()</code> , <code>update_password()</code>
Validation	User exists. Password satisfies security policy.
Permission	Super Admin, College Admin
Exceptions	<code>UserNotFoundException</code> , <code>ValidationException</code>
Logic	Validate request → Retrieve user → Hash new password → Update password → Return success response

delete_user()

Item	Details
Purpose	Soft delete a user account.

Item	Details
Repository	<code>get_user_by_id()</code> , <code>delete_user()</code>
Validation	User exists. User is not linked to active admission processes (if applicable).
Permission	Super Admin
Exceptions	<code>UserNotFoundException</code> , <code>UserInUseException</code>
Logic	Validate ID → Retrieve user → Verify dependencies → Soft delete user → Return success response

4. Module Notes

Database Tables

- `users`
- `roles`
- `institutions`

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User
- Password Hashing Utility

Implementation Notes

- Use UUID as the primary key.
- Store passwords in hashed format only.
- Never return password hashes in API responses.
- Enforce unique email and mobile number.
- Use soft delete with the `is_deleted` flag.

- Keep business logic in `service.py`.
- Keep database operations in `repository.py`.
- Record user creation, update, password reset, status change, and deletion in the Audit Logs module.
- Return standardized API responses using the common response utility.

Module 3 – Institutions

Module Information

Property	Value
Module Name	Institutions
Depends On	None
Used By	Users, Faculties, Departments, Courses, Admission Cycles, Students, Applications
Primary Table	<code>institutions</code>

1. Module Overview

Purpose

The **Institutions** module manages college and institution information within the Admitix system. It serves as the root entity for institution-specific data and provides organizational details to dependent modules.

Responsibilities

- Register institutions.
 - Retrieve institution information.
 - Update institution details.
 - Activate or deactivate institutions.
 - Manage institution records.
 - Provide institution data to dependent modules.
-

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/institutions</code>	<code>create_institution()</code>	Register a new institution
GET	<code>/institutions</code>	<code>get_institutions()</code>	Retrieve all institutions
GET	<code>/institutions/{id}</code>	<code>get_institution()</code>	Retrieve institution details
PUT	<code>/institutions/{id}</code>	<code>update_institution()</code>	Update institution information
PATCH	<code>/institutions/{id}/status</code>	<code>update_institution_status()</code>	Activate/Deactivate institution
DELETE	<code>/institutions/{id}</code>	<code>delete_institution()</code>	Soft delete institution

(Endpoints are based on your API specification.)

3. Service Implementation

create_institution()

Item	Details
Purpose	Register a new institution.
Repository	<code>get_institution_by_code()</code> , <code>get_institution_by_email()</code> , <code>create_institution()</code>
Validation	Institution name, code, email, and contact information are required. Institution code and email must be unique.
Permission	Super Admin
Exceptions	<code>InstitutionAlreadyExistsException</code> , <code>ValidationException</code>
Logic	Validate request → Check permission → Verify duplicate institution code/email → Create institution → Return success response

get_institutions()

Item	Details
Purpose	Retrieve all institutions.

Item	Details
Repository	<code>get_institutions()</code>
Validation	Validate pagination, filters, and sorting parameters.
Permission	Super Admin
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve institutions → Apply filters & pagination → Return response

get_institution()

Item	Details
Purpose	Retrieve institution details.
Repository	<code>get_institution_by_id()</code>
Validation	Validate Institution UUID.
Permission	Super Admin, College Admin
Exceptions	<code>InstitutionNotFoundException</code>
Logic	Validate ID → Retrieve institution → Return response

update_institution()

Item	Details
Purpose	Update institution information.
Repository	<code>get_institution_by_id()</code> , <code>get_institution_by_code()</code> , <code>update_institution()</code>
Validation	Institution exists. Updated institution code and email must remain unique.
Permission	Super Admin
Exceptions	<code>InstitutionNotFoundException</code> , <code>InstitutionAlreadyExistsException</code>
Logic	Validate request → Retrieve institution → Check duplicate institution code/email → Update institution → Return success response

update_institution_status()

Item	Details
Purpose	Activate or deactivate an institution.
Repository	<code>get_institution_by_id()</code> , <code>update_institution_status()</code>

Item	Details
Validation	Institution exists. Status value is valid.
Permission	Super Admin
Exceptions	<code>InstitutionNotFoundException</code> , <code>InvalidStatusException</code>
Logic	Validate ID → Retrieve institution → Update status → Return success response

delete_institution()

Item	Details
Purpose	Soft delete an institution.
Repository	<code>get_institution_by_id()</code> , <code>check_institution_dependencies()</code> , <code>delete_institution()</code>
Validation	Institution exists. Institution should not contain active users, departments, courses, or admission cycles.
Permission	Super Admin
Exceptions	<code>InstitutionNotFoundException</code> , <code>InstitutionInUseException</code>
Logic	Validate ID → Retrieve institution → Verify dependent records → Soft delete institution → Return success response

4. Module Notes

Database Tables

- `institutions`

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User

Module 4 – Faculties

Module Information

Property	Value
Module Name	Faculties
Depends On	Institutions
Used By	Departments, Courses, Students
Primary Table	<code>faculties</code>

1. Module Overview

Purpose

The **Faculties** module manages the faculties available within an institution (e.g., Engineering, Pharmacy, Medical, Law). It serves as the parent entity for departments and helps organize academic programs.

Responsibilities

- Create faculty records.
- Retrieve faculty information.
- Update faculty details.
- Activate or deactivate faculties.
- Provide faculty information to dependent modules.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/faculties</code>	<code>create_faculty()</code>	Create a new faculty
GET	<code>/faculties</code>	<code>get_faculties()</code>	Retrieve all faculties
GET	<code>/faculties/{id}</code>	<code>get_faculty()</code>	Retrieve faculty details
PUT	<code>/faculties/{id}</code>	<code>update_faculty()</code>	Update faculty information

Method	Endpoint	Service Function	Description
DELETE	<code>/faculties/{id}</code>	<code>delete_faculty()</code>	Soft delete a faculty

(Endpoints are based on your API specification.)

3. Service Implementation

create_faculty()

Item	Details
Purpose	Create a new faculty under an institution.
Repository	<code>get_institution_by_id()</code> , <code>get_faculty_by_name()</code> , <code>create_faculty()</code>
Validation	Institution must exist. Faculty name is required and must be unique within the institution.
Permission	Super Admin, College Admin
Exceptions	<code>InstitutionNotFoundException</code> , <code>FacultyAlreadyExistsException</code> , <code>ValidationException</code>
Logic	Validate request → Check permission → Verify institution → Check duplicate faculty → Create faculty → Return success response

get_faculties()

Item	Details
Purpose	Retrieve all faculties.
Repository	<code>get_faculties()</code>
Validation	Validate pagination, filters, and sorting parameters.
Permission	Super Admin, College Admin
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve faculties → Apply filters & pagination → Return response

get_faculty()

Item	Details
Purpose	Retrieve faculty details by ID.

Item	Details
Repository	<code>get_faculty_by_id()</code>
Validation	Validate Faculty UUID.
Permission	Super Admin, College Admin
Exceptions	<code>FacultyNotFoundException</code>
Logic	Validate ID → Retrieve faculty → Return response

update_faculty()

Item	Details
Purpose	Update faculty information.
Repository	<code>get_faculty_by_id()</code> , <code>get_faculty_by_name()</code> , <code>update_faculty()</code>
Validation	Faculty exists. Updated faculty name must remain unique within the institution.
Permission	Super Admin, College Admin
Exceptions	<code>FacultyNotFoundException</code> , <code>FacultyAlreadyExistsException</code>
Logic	Validate request → Retrieve faculty → Check duplicate faculty name → Update faculty → Return success response

delete_faculty()

Item	Details
Purpose	Soft delete an existing faculty.
Repository	<code>get_faculty_by_id()</code> , <code>check_faculty_dependencies()</code> , <code>delete_faculty()</code>
Validation	Faculty exists. Faculty should not have active departments or courses associated with it.
Permission	Super Admin
Exceptions	<code>FacultyNotFoundException</code> , <code>FacultyInUseException</code>
Logic	Validate ID → Retrieve faculty → Verify dependent records → Soft delete faculty → Return success response

4. Module Notes

Database Tables

- `faculties`

- `institutions` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User

Module 5 – Departments

Module Information

Property	Value
Module Name	Departments
Depends On	Institutions, Faculties
Used By	Courses, Students, Staff, Applications
Primary Table	<code>departments</code>

1. Module Overview

Purpose

The **Departments** module manages academic departments within an institution. Each department belongs to a faculty and serves as the parent entity for courses, students, staff, and admission-related activities.

Responsibilities

- Create department records.
- Retrieve department information.

- Update department details.
- Activate or deactivate departments.
- Provide department information to dependent modules.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/departments</code>	<code>create_department()</code>	Create a new department
GET	<code>/departments</code>	<code>get_departments()</code>	Retrieve all departments
GET	<code>/departments/{id}</code>	<code>get_department()</code>	Retrieve department details
PUT	<code>/departments/{id}</code>	<code>update_department()</code>	Update department information
DELETE	<code>/departments/{id}</code>	<code>delete_department()</code>	Soft delete a department

(Endpoints are based on your API specification.)

3. Service Implementation

create_department()

Item	Details
Purpose	Create a new department under a faculty.
Repository	<code>get_institution_by_id()</code> , <code>get_faculty_by_id()</code> , <code>get_department_by_name()</code> , <code>create_department()</code>
Validation	Institution and Faculty must exist. Department name is required and must be unique within the institution.
Permission	Super Admin, College Admin
Exceptions	<code>InstitutionNotFoundException</code> , <code>FacultyNotFoundException</code> , <code>DepartmentAlreadyExistsException</code> , <code>ValidationException</code>
Logic	Validate request → Check permission → Verify institution → Verify faculty → Check duplicate department → Create department → Return success response

get_departments()

Item	Details
Purpose	Retrieve all departments.
Repository	<code>get_departments()</code>
Validation	Validate pagination, filters, and sorting parameters.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve departments → Apply filters & pagination → Return response

get_department()

Item	Details
Purpose	Retrieve department details by ID.
Repository	<code>get_department_by_id()</code>
Validation	Validate Department UUID.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>DepartmentNotFoundException</code>
Logic	Validate ID → Retrieve department → Return response

update_department()

Item	Details
Purpose	Update department information.
Repository	<code>get_department_by_id()</code> , <code>get_department_by_name()</code> , <code>update_department()</code>
Validation	Department exists. Updated department name must remain unique within the institution.
Permission	Super Admin, College Admin
Exceptions	<code>DepartmentNotFoundException</code> , <code>DepartmentAlreadyExistsException</code>
Logic	Validate request → Retrieve department → Check duplicate department name → Update department → Return success response

delete_department()

Item	Details
Purpose	Soft delete an existing department.
Repository	<code>get_department_by_id()</code> , <code>check_department_dependencies()</code> , <code>delete_department()</code>
Validation	Department exists. Department should not have active courses, students, or staff associated with it.
Permission	Super Admin
Exceptions	<code>DepartmentNotFoundException</code> , <code>DepartmentInUseException</code>
Logic	Validate ID → Retrieve department → Verify dependent records → Soft delete department → Return success response

4. Module Notes

Database Tables

- `departments`
- `institutions` (Reference Validation)
- `faculties` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User

Module 6 – Courses

Module Information

Property	Value
Module Name	Courses
Depends On	Institutions, Faculties, Departments
Used By	Admission Cycles, Students, Applications, Seat Matrix
Primary Table	<code>courses</code>

1. Module Overview

Purpose

The **Courses** module manages all academic courses offered by an institution. Each course is associated with a department and faculty and serves as the foundation for admission cycles, seat allocation, and student admissions.

Responsibilities

- Create course records.
- Retrieve course information.
- Update course details.
- Activate or deactivate courses.
- Provide course information to dependent modules.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/courses</code>	<code>create_course()</code>	Create a new course
GET	<code>/courses</code>	<code>get_courses()</code>	Retrieve all courses
GET	<code>/courses/{id}</code>	<code>get_course()</code>	Retrieve course details
PUT	<code>/courses/{id}</code>	<code>update_course()</code>	Update course information
DELETE	<code>/courses/{id}</code>	<code>delete_course()</code>	Soft delete a course

(Endpoints are based on your API specification.)

3. Service Implementation

create_course()

Item	Details
Purpose	Create a new academic course under a department.
Repository	<code>get_department_by_id()</code> , <code>get_course_by_name()</code> , <code>create_course()</code>
Validation	Department must exist. Course name and code are required. Course code must be unique within the institution.
Permission	Super Admin, College Admin
Exceptions	<code>DepartmentNotFoundException</code> , <code>CourseAlreadyExistsException</code> , <code>ValidationException</code>
Logic	Validate request → Check permission → Verify department → Check duplicate course → Create course → Return success response

get_courses()

Item	Details
Purpose	Retrieve all courses.
Repository	<code>get_courses()</code>
Validation	Validate pagination, filters, sorting, and department filter.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve courses → Apply filters & pagination → Return response

get_course()

Item	Details
Purpose	Retrieve course details by ID.
Repository	<code>get_course_by_id()</code>
Validation	Validate Course UUID.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>CourseNotFoundException</code>
Logic	Validate ID → Retrieve course → Return response

update_course()

Item	Details
Purpose	Update course information.
Repository	<code>get_course_by_id()</code> , <code>get_course_by_code()</code> , <code>update_course()</code>
Validation	Course exists. Updated course code must remain unique.
Permission	Super Admin, College Admin
Exceptions	<code>CourseNotFoundException</code> , <code>CourseAlreadyExistsException</code>
Logic	Validate request → Retrieve course → Check duplicate course code → Update course → Return success response

delete_course()

Item	Details
Purpose	Soft delete an existing course.
Repository	<code>get_course_by_id()</code> , <code>check_course_dependencies()</code> , <code>delete_course()</code>
Validation	Course exists. Course should not have active admission cycles, students, applications, or seat matrix records.
Permission	Super Admin
Exceptions	<code>CourseNotFoundException</code> , <code>CourseInUseException</code>
Logic	Validate ID → Retrieve course → Verify dependent records → Soft delete course → Return success response

4. Module Notes

Database Tables

- `courses`
- `departments` (Reference Validation)
- `faculties` (Reference Validation)
- `institutions` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`

- `common/validators.py`

Dependencies

- Database Session
- Current User

Module 7 – Admission Cycles

Module Information

Property	Value
Module Name	Admission Cycles
Depends On	Institutions, Courses
Used By	Applications, Students, Seat Matrix, Payments
Primary Table	<code>admission_cycles</code>

1. Module Overview

Purpose

The **Admission Cycles** module manages admission sessions for institutions and courses. It defines the admission period, controls whether admissions are open or closed, and serves as the entry point for application processing.

Responsibilities

- Create admission cycles.
- Retrieve admission cycle information.
- Update admission cycle details.
- Open or close admission cycles.
- Provide active admission cycle information to dependent modules.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/admission-cycles</code>	<code>create_admission_cycle()</code>	Create a new admission cycle
GET	<code>/admission-cycles</code>	<code>get_admission_cycles()</code>	Retrieve all admission cycles
GET	<code>/admission-cycles/{id}</code>	<code>get_admission_cycle()</code>	Retrieve admission cycle details
PUT	<code>/admission-cycles/{id}</code>	<code>update_admission_cycle()</code>	Update admission cycle
PATCH	<code>/admission-cycles/{id}/status</code>	<code>update_admission_cycle_status()</code>	Open, close, or update admission cycle status
DELETE	<code>/admission-cycles/{id}</code>	<code>delete_admission_cycle()</code>	Soft delete an admission cycle

(Endpoints are based on your API specification.)

3. Service Implementation

create_admission_cycle()

Item	Details
Purpose	Create a new admission cycle for a course.
Repository	<code>get_institution_by_id()</code> , <code>get_course_by_id()</code> , <code>get_admission_cycle()</code> , <code>create_admission_cycle()</code>
Validation	Institution and Course must exist. Academic year, admission dates, and cycle name are required. Admission period must be valid.
Permission	Super Admin, College Admin
Exceptions	<code>InstitutionNotFoundException</code> , <code>CourseNotFoundException</code> , <code>AdmissionCycleAlreadyExistsException</code> , <code>ValidationException</code>
Logic	Validate request → Check permission → Verify institution → Verify course → Check duplicate cycle → Create admission cycle → Return success response

get_admission_cycles()

Item	Details
Purpose	Retrieve all admission cycles.

Item	Details
Repository	<code>get_admission_cycles()</code>
Validation	Validate pagination, filters, sorting, academic year, and status filters.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve admission cycles → Apply filters & pagination → Return response

get_admission_cycle()

Item	Details
Purpose	Retrieve admission cycle details by ID.
Repository	<code>get_admission_cycle_by_id()</code>
Validation	Validate Admission Cycle UUID.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>AdmissionCycleNotFoundException</code>
Logic	Validate ID → Retrieve admission cycle → Return response

update_admission_cycle()

Item	Details
Purpose	Update admission cycle information.
Repository	<code>get_admission_cycle_by_id()</code> , <code>update_admission_cycle()</code>
Validation	Admission cycle exists. Updated admission dates must be valid.
Permission	Super Admin, College Admin
Exceptions	<code>AdmissionCycleNotFoundException</code> , <code>ValidationException</code>
Logic	Validate request → Retrieve admission cycle → Validate updated information → Update admission cycle → Return success response

update_admission_cycle_status()

Item	Details
Purpose	Open, close, or update the admission cycle status.
Repository	<code>get_admission_cycle_by_id()</code> , <code>update_admission_cycle_status()</code>

Item	Details
Validation	Admission cycle exists. Status transition must be valid.
Permission	Super Admin, College Admin
Exceptions	<code>AdmissionCycleNotFoundException</code> , <code>InvalidStatusException</code>
Logic	Validate ID → Retrieve admission cycle → Validate status transition → Update status → Return success response

delete_admission_cycle()

Item	Details
Purpose	Soft delete an admission cycle.
Repository	<code>get_admission_cycle_by_id()</code> , <code>check_cycle_dependencies()</code> , <code>delete_admission_cycle()</code>
Validation	Admission cycle exists. No active applications or seat allocations should be associated with it.
Permission	Super Admin
Exceptions	<code>AdmissionCycleNotFoundException</code> , <code>AdmissionCycleInUseException</code>
Logic	Validate ID → Retrieve admission cycle → Verify dependent records → Soft delete admission cycle → Return success response

4. Module Notes

Database Tables

- `admission_cycles`
- `institutions` (Reference Validation)
- `courses` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User

Module 8 – Staff

Module Information

Property	Value
Module Name	Staff
Depends On	Users, Institutions, Departments
Used By	Admission Cycles, Applications, Documents, AI Verification, Audit Logs
Primary Table	<code>staff</code>

1. Module Overview

Purpose

The **Staff** module manages institution staff members involved in the admission process. It stores staff information, assigns departments and roles, and enables staff members to perform admission-related operations.

Responsibilities

- Register staff members.
- Retrieve staff information.
- Update staff details.
- Activate or deactivate staff.
- Assign staff to departments.
- Provide staff information to dependent modules.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/staff</code>	<code>create_staff()</code>	Register a new staff member
GET	<code>/staff</code>	<code>get_staff()</code>	Retrieve all staff members
GET	<code>/staff/{id}</code>	<code>get_staff_by_id()</code>	Retrieve staff details
PUT	<code>/staff/{id}</code>	<code>update_staff()</code>	Update staff information
PATCH	<code>/staff/{id}/status</code>	<code>update_staff_status()</code>	Activate/Deactivate staff
DELETE	<code>/staff/{id}</code>	<code>delete_staff()</code>	Soft delete staff member

(Endpoints are based on your API specification.)

3. Service Implementation

create_staff()

Item	Details
Purpose	Register a new staff member for an institution.
Repository	<code>get_user_by_id()</code> , <code>get_department_by_id()</code> , <code>get_staff_by_employee_code()</code> , <code>create_staff()</code>
Validation	User and Department must exist. Employee code must be unique. Required fields must be provided.
Permission	Super Admin, College Admin
Exceptions	<code>UserNotFoundException</code> , <code>DepartmentNotFoundException</code> , <code>StaffAlreadyExistsException</code> , <code>ValidationException</code>
Logic	Validate request → Check permission → Verify user → Verify department → Check duplicate employee code → Create staff record → Return success response

get_staff()

Item	Details
Purpose	Retrieve all staff members.
Repository	<code>get_staff()</code>
Validation	Validate pagination, filters, sorting, and department filter.
Permission	Super Admin, College Admin
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve staff → Apply filters & pagination → Return response

get_staff_by_id()

Item	Details
Purpose	Retrieve staff details by ID.
Repository	<code>get_staff_by_id()</code>
Validation	Validate Staff UUID.
Permission	Super Admin, College Admin
Exceptions	<code>StaffNotFoundException</code>
Logic	Validate ID → Retrieve staff → Return response

update_staff()

Item	Details
Purpose	Update staff information.
Repository	<code>get_staff_by_id()</code> , <code>update_staff()</code>
Validation	Staff exists. Updated employee code must remain unique.
Permission	Super Admin, College Admin
Exceptions	<code>StaffNotFoundException</code> , <code>StaffAlreadyExistsException</code>
Logic	Validate request → Retrieve staff → Validate updates → Update staff → Return success response

update_staff_status()

Item	Details
Purpose	Activate or deactivate a staff member.
Repository	<code>get_staff_by_id()</code> , <code>update_staff_status()</code>
Validation	Staff exists. Status value is valid.
Permission	Super Admin, College Admin
Exceptions	<code>StaffNotFoundException</code> , <code>InvalidStatusException</code>
Logic	Validate ID → Retrieve staff → Update status → Return success response

delete_staff()

Item	Details
Purpose	Soft delete a staff member.
Repository	<code>get_staff_by_id()</code> , <code>check_staff_dependencies()</code> , <code>delete_staff()</code>
Validation	Staff exists. Staff should not have pending application reviews, document verifications, or assigned admission tasks.
Permission	Super Admin
Exceptions	<code>StaffNotFoundException</code> , <code>StaffInUseException</code>
Logic	Validate ID → Retrieve staff → Verify dependent records → Soft delete staff → Return success response

4. Module Notes

Database Tables

- `staff`
- `users` (Reference Validation)
- `departments` (Reference Validation)
- `institutions` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User

Module 9 – Students

Module Information

Property	Value
Module Name	Students
Depends On	Users, Institutions, Departments, Courses, Admission Cycles
Used By	Education Details, Entrance Exam Scores, Applications, Documents, Payments, Notifications
Primary Table	<code>students</code>

1. Module Overview

Purpose

The **Students** module manages student profiles participating in the admission process. It stores personal, academic, and contact information and serves as the central module for all admission-related activities.

Responsibilities

- Register student profiles.
- Retrieve student information.
- Update student details.
- Activate or deactivate student records.
- Link students with applications and supporting modules.
- Provide student information to dependent modules.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/students</code>	<code>create_student()</code>	Register a new student
GET	<code>/students</code>	<code>get_students()</code>	Retrieve all students
GET	<code>/students/{id}</code>	<code>get_student()</code>	Retrieve student details
PUT	<code>/students/{id}</code>	<code>update_student()</code>	Update student information
PATCH	<code>/students/{id}/status</code>	<code>update_student_status()</code>	Activate/Deactivate student
DELETE	<code>/students/{id}</code>	<code>delete_student()</code>	Soft delete student

(Endpoints are based on your API specification.)

3. Service Implementation

create_student()

Item	Details
Purpose	Register a new student profile.
Repository	<code>get_user_by_id()</code> , <code>get_course_by_id()</code> , <code>get_admission_cycle_by_id()</code> , <code>get_student_by_prn()</code> , <code>create_student()</code>
Validation	User, Course, and Admission Cycle must exist. Required fields must be provided. PRN/Enrollment Number must be unique (if applicable).
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>UserNotFoundException</code> , <code>CourseNotFoundException</code> , <code>AdmissionCycleNotFoundException</code> , <code>StudentAlreadyExistsException</code> , <code>ValidationException</code>
Logic	Validate request → Check permission → Verify user → Verify course → Verify admission cycle → Check duplicate student → Create student → Return success response

get_students()

Item	Details
Purpose	Retrieve all student records.
Repository	<code>get_students()</code>
Validation	Validate pagination, filters, sorting, department, course, and admission cycle filters.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve students → Apply filters & pagination → Return response

get_student()

Item	Details
Purpose	Retrieve student details by ID.
Repository	<code>get_student_by_id()</code>

Item	Details
Validation	Validate Student UUID.
Permission	Super Admin, College Admin, Admission Officer, Student (Own Profile)
Exceptions	<code>StudentNotFoundException</code>
Logic	Validate ID → Retrieve student → Return response

update_student()

Item	Details
Purpose	Update student information.
Repository	<code>get_student_by_id()</code> , <code>update_student()</code>
Validation	Student exists. Updated PRN/Enrollment Number must remain unique (if applicable).
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>StudentNotFoundException</code> , <code>StudentAlreadyExistsException</code>
Logic	Validate request → Retrieve student → Validate updates → Update student → Return success response

update_student_status()

Item	Details
Purpose	Activate or deactivate a student record.
Repository	<code>get_student_by_id()</code> , <code>update_student_status()</code>
Validation	Student exists. Status value is valid.
Permission	Super Admin, College Admin
Exceptions	<code>StudentNotFoundException</code> , <code>InvalidStatusException</code>
Logic	Validate ID → Retrieve student → Update status → Return success response

delete_student()

Item	Details
Purpose	Soft delete a student record.
Repository	<code>get_student_by_id()</code> , <code>check_student_dependencies()</code> , <code>delete_student()</code>
Validation	Student exists. Student should not have active applications, payments, or document verification processes.

Item	Details
Permission	Super Admin
Exceptions	<code>StudentNotFoundException</code> , <code>StudentInUseException</code>
Logic	Validate ID → Retrieve student → Verify dependent records → Soft delete student → Return success response

4. Module Notes

Database Tables

- `students`
- `users` (Reference Validation)
- `courses` (Reference Validation)
- `departments` (Reference Validation)
- `admission_cycles` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User

Module 10 – Education Details

Module Information

Property	Value
Module Name	Education Details
Depends On	Students

Property	Value
Used By	Applications, AI Verification
Primary Table	<code>education_details</code>

1. Module Overview

Purpose

The **Education Details** module manages the academic history of students. It stores qualification details required during the admission process and provides educational information to applications and document verification modules.

Responsibilities

- Store student academic records.
- Retrieve education details.
- Update qualification information.
- Validate educational records.
- Provide academic information to dependent modules.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/education-details</code>	<code>create_education_detail()</code>	Add education details
GET	<code>/education-details</code>	<code>get_education_details()</code>	Retrieve all education details
GET	<code>/education-details/{id}</code>	<code>get_education_detail()</code>	Retrieve education detail by ID
PUT	<code>/education-details/{id}</code>	<code>update_education_detail()</code>	Update education details
DELETE	<code>/education-details/{id}</code>	<code>delete_education_detail()</code>	Soft delete education details

(Endpoints are based on your API specification.)

3. Service Implementation

create_education_detail()

Item	Details
Purpose	Add educational qualification details for a student.
Repository	<code>get_student_by_id()</code> , <code>create_education_detail()</code>
Validation	Student must exist. Qualification, board/university, passing year, marks/CGPA, and institute name are required.
Permission	Student, Admission Officer, College Admin
Exceptions	<code>StudentNotFoundException</code> , <code>ValidationException</code>
Logic	Validate request → Verify student → Save education details → Return success response

get_education_details()

Item	Details
Purpose	Retrieve education records.
Repository	<code>get_education_details()</code>
Validation	Validate pagination, filters, and sorting parameters.
Permission	Super Admin, College Admin, Admission Officer, Student (Own Records)
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve records → Apply filters & pagination → Return response

get_education_detail()

Item	Details
Purpose	Retrieve a specific education record.
Repository	<code>get_education_detail_by_id()</code>
Validation	Validate Education Detail UUID.
Permission	Super Admin, College Admin, Admission Officer, Student (Own Record)
Exceptions	<code>EducationDetailNotFoundException</code>
Logic	Validate ID → Retrieve education detail → Return response

update_education_detail()

Item	Details
Purpose	Update an existing education record.
Repository	<code>get_education_detail_by_id()</code> , <code>update_education_detail()</code>
Validation	Education record exists. Updated academic information must be valid.
Permission	Student, Admission Officer, College Admin
Exceptions	<code>EducationDetailNotFoundException</code> , <code>ValidationException</code>
Logic	Validate request → Retrieve education detail → Validate updates → Update record → Return success response

delete_education_detail()

Item	Details
Purpose	Soft delete an education record.
Repository	<code>get_education_detail_by_id()</code> , <code>delete_education_detail()</code>
Validation	Education record exists.
Permission	College Admin, Super Admin
Exceptions	<code>EducationDetailNotFoundException</code>
Logic	Validate ID → Retrieve education detail → Soft delete record → Return success response

4. Module Notes

Database Tables

- `education_details`
- `students` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User

Module 11 – Entrance Exam Scores

Module Information

Property	Value
Module Name	Entrance Exam Scores
Depends On	Students
Used By	Applications, Merit Processing, Reports
Primary Table	entrance_exam_scores

1. Module Overview

Purpose

The **Entrance Exam Scores** module manages entrance examination details and scores submitted by students. It stores exam information required during the admission process and provides score data for applications, merit processing, and reporting.

Responsibilities

- Store entrance exam details.
- Retrieve exam score information.
- Update exam records.
- Validate score information.
- Provide exam scores to dependent modules.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	/entrance-exam-scores	create_exam_score()	Add entrance exam score
GET	/entrance-exam-scores	get_exam_scores()	Retrieve all exam scores
GET	/entrance-exam-scores/{id}	get_exam_score()	Retrieve exam score details
PUT	/entrance-exam-scores/{id}	update_exam_score()	Update exam score
DELETE	/entrance-exam-scores/{id}	delete_exam_score()	Soft delete exam score

(Endpoints are based on your API specification.)

3. Service Implementation

create_exam_score()

Item	Details
Purpose	Add entrance examination details for a student.
Repository	get_student_by_id() , create_exam_score()
Validation	Student must exist. Exam name, roll number, score/rank, and examination year are required.
Permission	Student, Admission Officer, College Admin
Exceptions	StudentNotFoundException , ValidationException
Logic	Validate request → Verify student → Save exam score → Return success response

get_exam_scores()

Item	Details
Purpose	Retrieve entrance exam scores.
Repository	get_exam_scores()
Validation	Validate pagination, filters, sorting, exam type, and year filters.
Permission	Super Admin, College Admin, Admission Officer, Student (Own Records)
Exceptions	ValidationException
Logic	Validate request → Retrieve exam scores → Apply filters & pagination → Return response

get_exam_score()

Item	Details
Purpose	Retrieve a specific entrance exam score.
Repository	<code>get_exam_score_by_id()</code>
Validation	Validate Entrance Exam Score UUID.
Permission	Super Admin, College Admin, Admission Officer, Student (Own Record)
Exceptions	<code>EntranceExamScoreNotFoundException</code>
Logic	Validate ID → Retrieve exam score → Return response

update_exam_score()

Item	Details
Purpose	Update an existing entrance exam score.
Repository	<code>get_exam_score_by_id()</code> , <code>update_exam_score()</code>
Validation	Exam record exists. Updated score, rank, and exam details must be valid.
Permission	Student, Admission Officer, College Admin
Exceptions	<code>EntranceExamScoreNotFoundException</code> , <code>ValidationException</code>
Logic	Validate request → Retrieve exam record → Validate updates → Update record → Return success response

delete_exam_score()

Item	Details
Purpose	Soft delete an entrance exam score.
Repository	<code>get_exam_score_by_id()</code> , <code>delete_exam_score()</code>
Validation	Exam record exists.
Permission	College Admin, Super Admin
Exceptions	<code>EntranceExamScoreNotFoundException</code>
Logic	Validate ID → Retrieve exam record → Soft delete record → Return success response

4. Module Notes

Database Tables

- `entrance_exam_scores`
- `students` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User

Module 12 – Applications

Module Information

Property	Value
Module Name	Applications
Depends On	Students, Admission Cycles, Courses
Used By	Application Preferences, Documents, AI Verification, Payments, Application Status History, Notifications
Primary Table	<code>applications</code>

1. Module Overview

Purpose

The **Applications** module manages student admission applications. It serves as the central module of the admission workflow by creating, updating, tracking, and processing student applications throughout the admission cycle.

Responsibilities

- Create admission applications.
- Retrieve application information.
- Update application details.
- Submit applications for verification.
- Track application status.
- Provide application data to dependent modules.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/applications</code>	<code>create_application()</code>	Create a new application
GET	<code>/applications</code>	<code>get_applications()</code>	Retrieve all applications
GET	<code>/applications/{id}</code>	<code>get_application()</code>	Retrieve application details
PUT	<code>/applications/{id}</code>	<code>update_application()</code>	Update application
PATCH	<code>/applications/{id}/submit</code>	<code>submit_application()</code>	Submit application
PATCH	<code>/applications/{id}/status</code>	<code>update_application_status()</code>	Update application status
DELETE	<code>/applications/{id}</code>	<code>delete_application()</code>	Soft delete application

(Endpoints are based on your API specification.)

3. Service Implementation

create_application()

Item	Details
Purpose	Create a new admission application for a student.
Repository	<code>get_student_by_id()</code> , <code>get_admission_cycle_by_id()</code> , <code>get_course_by_id()</code> , <code>get_application()</code> , <code>create_application()</code>

Item	Details
Validation	Student, Course, and Admission Cycle must exist. Only one active application is allowed per admission cycle.
Permission	Student, Admission Officer, College Admin
Exceptions	<code>StudentNotFoundException</code> , <code>CourseNotFoundException</code> , <code>AdmissionCycleNotFoundException</code> , <code>ApplicationAlreadyExistsException</code> , <code>ValidationException</code>
Logic	Validate request → Verify student → Verify admission cycle → Verify course → Check duplicate application → Create application → Return success response

get_applications()

Item	Details
Purpose	Retrieve all applications.
Repository	<code>get_applications()</code>
Validation	Validate pagination, filters, sorting, admission cycle, course, department, and status filters.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve applications → Apply filters & pagination → Return response

get_application()

Item	Details
Purpose	Retrieve application details.
Repository	<code>get_application_by_id()</code>
Validation	Validate Application UUID.
Permission	Super Admin, College Admin, Admission Officer, Student (Own Application)
Exceptions	<code>ApplicationNotFoundException</code>
Logic	Validate ID → Retrieve application → Return response

update_application()

Item	Details
Purpose	Update an application before submission.
Repository	<code>get_application_by_id()</code> , <code>update_application()</code>
Validation	Application exists. Application must not be locked or already approved/rejected.
Permission	Student (Own Application), Admission Officer
Exceptions	<code>ApplicationNotFoundException</code> , <code>ApplicationLockedException</code> , <code>ValidationException</code>
Logic	Validate request → Retrieve application → Verify editable status → Update application → Return success response

submit_application()

Item	Details
Purpose	Submit an application for verification.
Repository	<code>get_application_by_id()</code> , <code>update_application_status()</code>
Validation	Application exists. Mandatory information and required documents must be completed before submission.
Permission	Student
Exceptions	<code>ApplicationNotFoundException</code> , <code>IncompleteApplicationException</code>
Logic	Validate application → Verify required information → Verify mandatory documents → Update status to Submitted → Return success response

update_application_status()

Item	Details
Purpose	Update the application status during admission processing.
Repository	<code>get_application_by_id()</code> , <code>update_application_status()</code>
Validation	Application exists. Status transition must follow the admission workflow.
Permission	Admission Officer, College Admin
Exceptions	<code>ApplicationNotFoundException</code> , <code>InvalidStatusException</code>
Logic	Validate ID → Retrieve application → Validate status transition → Update status → Record status history → Return success response

delete_application()

Item	Details
Purpose	Soft delete an application.
Repository	<code>get_application_by_id()</code> , <code>check_application_dependencies()</code> , <code>delete_application()</code>
Validation	Application exists. Submitted, verified, or approved applications cannot be deleted.
Permission	Super Admin, College Admin
Exceptions	<code>ApplicationNotFoundException</code> , <code>ApplicationInUseException</code>
Logic	Validate ID → Retrieve application → Verify deletion eligibility → Soft delete application → Return success response

4. Module Notes

Database Tables

- `applications`
- `students` (Reference Validation)
- `courses` (Reference Validation)
- `admission_cycles` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Module 13 – Application Preferences

Module Information

Property	Value
Module Name	Application Preferences
Depends On	Applications, Courses

Property	Value
Used By	Admission Processing, Reports
Primary Table	<code>application_preferences</code>

1. Module Overview

Purpose

The **Application Preferences** module manages the course preferences selected by students during the admission process. It stores preference order and provides preference data for admission and seat allocation.

Responsibilities

- Add course preferences.
- Retrieve preference details.
- Update preference order.
- Remove preferences.
- Provide preference data for admission processing.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/application-preferences</code>	<code>create_preference()</code>	Add application preference
GET	<code>/application-preferences</code>	<code>get_preferences()</code>	Retrieve all preferences
GET	<code>/application-preferences/{id}</code>	<code>get_preference()</code>	Retrieve preference details
PUT	<code>/application-preferences/{id}</code>	<code>update_preference()</code>	Update preference
DELETE	<code>/application-preferences/{id}</code>	<code>delete_preference()</code>	Remove preference

(Endpoints are based on your API specification.)

3. Service Implementation

create_preference()

Item	Details
Purpose	Add a course preference to an application.
Repository	<code>get_application_by_id()</code> , <code>get_course_by_id()</code> , <code>get_preference()</code> , <code>create_preference()</code>
Validation	Application and Course must exist. Preference order must be unique within the application.
Permission	Student, Admission Officer
Exceptions	<code>ApplicationNotFoundException</code> , <code>CourseNotFoundException</code> , <code>PreferenceAlreadyExistsException</code> , <code>ValidationException</code>
Logic	Validate request → Verify application → Verify course → Check duplicate preference → Save preference → Return success response

get_preferences()

Item	Details
Purpose	Retrieve application preferences.
Repository	<code>get_preferences()</code>
Validation	Validate pagination, filters, sorting, and application filter.
Permission	Super Admin, College Admin, Admission Officer, Student (Own Application)
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve preferences → Apply filters & pagination → Return response

get_preference()

Item	Details
Purpose	Retrieve preference details by ID.
Repository	<code>get_preference_by_id()</code>
Validation	Validate Preference UUID.
Permission	Super Admin, College Admin, Admission Officer, Student (Own Application)
Exceptions	<code>PreferenceNotFoundException</code>

Item	Details
Logic	Validate ID → Retrieve preference → Return response

update_preference()

Item	Details
Purpose	Update course preference order.
Repository	<code>get_preference_by_id()</code> , <code>update_preference()</code>
Validation	Preference exists. Preference order must remain unique within the application.
Permission	Student, Admission Officer
Exceptions	<code>PreferenceNotFoundException</code> , <code>ValidationException</code>
Logic	Validate request → Retrieve preference → Validate order → Update preference → Return success response

delete_preference()

Item	Details
Purpose	Remove a course preference.
Repository	<code>get_preference_by_id()</code> , <code>delete_preference()</code>
Validation	Preference exists. Preferences cannot be deleted after application submission.
Permission	Student, Admission Officer
Exceptions	<code>PreferenceNotFoundException</code> , <code>ApplicationLockedException</code>
Logic	Validate ID → Retrieve preference → Verify application status → Delete preference → Return success response

4. Module Notes

Database Tables

- `application_preferences`
- `applications` (Reference Validation)
- `courses` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User

Module 14 – Document Types

Module Information

Property	Value
Module Name	Document Types
Depends On	Institutions, Courses, Admission Cycles
Used By	Documents, AI Verification
Primary Table	<code>document_types</code>

1. Module Overview

Purpose

The **Document Types** module manages the list of documents required for admission. It defines mandatory and optional documents for different admission cycles and courses.

Responsibilities

- Create document types.
- Retrieve document type information.
- Update document requirements.
- Activate or deactivate document types.
- Provide document configuration to dependent modules.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	/document-types	create_document_type()	Create a document type
GET	/document-types	get_document_types()	Retrieve all document types
GET	/document-types/{id}	get_document_type()	Retrieve document type details
PUT	/document-types/{id}	update_document_type()	Update document type
DELETE	/document-types/{id}	delete_document_type()	Soft delete document type

(Endpoints are based on your API specification.)

3. Service Implementation

create_document_type()

Item	Details
Purpose	Create a new document type for admissions.
Repository	get_document_type_by_name() , create_document_type()
Validation	Document name is required and must be unique. Mandatory flag and supported file types must be valid.
Permission	Super Admin, College Admin
Exceptions	DocumentTypeAlreadyExistsException , ValidationException
Logic	Validate request → Check permission → Verify duplicate document type → Create document type → Return success response

get_document_types()

Item	Details
Purpose	Retrieve all document types.
Repository	get_document_types()
Validation	Validate pagination, filters, sorting, and status filters.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	ValidationException
Logic	Validate request → Retrieve document types → Apply filters & pagination → Return response

get_document_type()

Item	Details
Purpose	Retrieve document type details.
Repository	<code>get_document_type_by_id()</code>
Validation	Validate Document Type UUID.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>DocumentTypeNotFoundException</code>
Logic	Validate ID → Retrieve document type → Return response

update_document_type()

Item	Details
Purpose	Update document type information.
Repository	<code>get_document_type_by_id()</code> , <code>get_document_type_by_name()</code> , <code>update_document_type()</code>
Validation	Document type exists. Updated name must remain unique.
Permission	Super Admin, College Admin
Exceptions	<code>DocumentTypeNotFoundException</code> , <code>DocumentTypeAlreadyExistsException</code>
Logic	Validate request → Retrieve document type → Check duplicate name → Update document type → Return success response

delete_document_type()

Item	Details
Purpose	Soft delete a document type.
Repository	<code>get_document_type_by_id()</code> , <code>check_document_type_usage()</code> , <code>delete_document_type()</code>
Validation	Document type exists. It should not be linked to active document records.
Permission	Super Admin
Exceptions	<code>DocumentTypeNotFoundException</code> , <code>DocumentTypeInUseException</code>
Logic	Validate ID → Retrieve document type → Verify dependencies → Soft delete document type → Return success response

4. Module Notes

Database Tables

- `document_types`

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User

Module 15 – Documents

Module Information

Property	Value
Module Name	Documents
Depends On	Students, Applications, Document Types
Used By	AI Verification, Admission Officer, Applications
Primary Table	<code>documents</code>

1. Module Overview

Purpose

The **Documents** module manages document uploads submitted by students during the admission process. It stores document metadata, upload status, and verification status, making documents available for AI and manual verification.

Responsibilities

- Upload student documents.
- Retrieve uploaded documents.
- Update document information.
- Download/view documents.
- Delete documents.
- Provide documents for verification.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/documents</code>	<code>upload_document()</code>	Upload a document
GET	<code>/documents</code>	<code>get_documents()</code>	Retrieve all documents
GET	<code>/documents/{id}</code>	<code>get_document()</code>	Retrieve document details
PUT	<code>/documents/{id}</code>	<code>update_document()</code>	Update document information
GET	<code>/documents/{id}/download</code>	<code>download_document()</code>	Download/View document
DELETE	<code>/documents/{id}</code>	<code>delete_document()</code>	Soft delete document

(Endpoints are based on your API specification.)

3. Service Implementation

upload_document()

Item	Details
Purpose	Upload a document for an application.
Repository	<code>get_application_by_id()</code> , <code>get_document_type_by_id()</code> , <code>create_document()</code>
Validation	Application and Document Type must exist. File type, size, and document format must be valid.
Permission	Student
Exceptions	<code>ApplicationNotFoundException</code> , <code>DocumentTypeNotFoundException</code> , <code>InvalidFileException</code> , <code>ValidationException</code>

Item	Details
Logic	Validate request → Verify application → Verify document type → Validate file → Upload file → Save metadata → Return success response

get_documents()

Item	Details
Purpose	Retrieve uploaded documents.
Repository	<code>get_documents()</code>
Validation	Validate pagination, filters, sorting, document type, and verification status filters.
Permission	Super Admin, College Admin, Admission Officer, Student (Own Documents)
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve documents → Apply filters & pagination → Return response

get_document()

Item	Details
Purpose	Retrieve document details.
Repository	<code>get_document_by_id()</code>
Validation	Validate Document UUID.
Permission	Super Admin, College Admin, Admission Officer, Student (Own Document)
Exceptions	<code>DocumentNotFoundException</code>
Logic	Validate ID → Retrieve document → Return response

update_document()

Item	Details
Purpose	Replace or update document information.
Repository	<code>get_document_by_id()</code> , <code>update_document()</code>
Validation	Document exists. Updated file must satisfy upload rules.
Permission	Student, Admission Officer
Exceptions	<code>DocumentNotFoundException</code> , <code>InvalidFileException</code> , <code>ValidationException</code>

Item	Details
Logic	Validate request → Retrieve document → Validate new file → Update document → Return success response

download_document()

Item	Details
Purpose	Download or view an uploaded document.
Repository	<code>get_document_by_id()</code>
Validation	Document exists and user has access permission.
Permission	Student (Own Document), Admission Officer, College Admin
Exceptions	<code>DocumentNotFoundException</code> , <code>AccessDeniedException</code>
Logic	Validate ID → Verify permission → Retrieve document → Return file response

delete_document()

Item	Details
Purpose	Soft delete a document.
Repository	<code>get_document_by_id()</code> , <code>delete_document()</code>
Validation	Document exists. Verified documents cannot be deleted without authorization.
Permission	Student, College Admin
Exceptions	<code>DocumentNotFoundException</code> , <code>DocumentLockedException</code>
Logic	Validate ID → Retrieve document → Verify deletion rules → Soft delete document → Return success response

4. Module Notes

Database Tables

- `documents`
- `applications` (Reference Validation)
- `document_types` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User
- File Storage Utility

Module 16 – AI Verification

Module Information

Property	Value
Module Name	AI Verification
Depends On	Applications, Documents
Used By	Applications, Admission Officer, Audit Logs
Primary Table	<code>ai_verifications</code>

1. Module Overview

Purpose

The **AI Verification** module manages automated document verification. It validates uploaded documents using AI services, detects missing or invalid documents, and stores verification results for manual review.

Responsibilities

- Verify uploaded documents.
- Store AI verification results.
- Retrieve verification reports.

- Re-run document verification.
- Forward results for manual verification.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/ai-verification/{application_id}</code>	<code>verify_documents()</code>	Start AI verification
GET	<code>/ai-verification</code>	<code>get_verifications()</code>	Retrieve verification records
GET	<code>/ai-verification/{id}</code>	<code>get_verification()</code>	Retrieve verification details
POST	<code>/ai-verification/{id}/retry</code>	<code>retry_verification()</code>	Re-run AI verification
PATCH	<code>/ai-verification/{id}/status</code>	<code>update_verification_status()</code>	Update verification status

(Endpoints are based on your API specification.)

3. Service Implementation

verify_documents()

Item	Details
Purpose	Perform AI verification of all documents submitted for an application.
Repository	<code>get_application_by_id()</code> , <code>get_documents_by_application()</code> , <code>create_verification()</code>
Validation	Application exists. Required documents are uploaded.
Permission	Admission Officer, College Admin
Exceptions	<code>ApplicationNotFoundException</code> , <code>DocumentNotFoundException</code> , <code>ValidationException</code>
Logic	Validate request → Retrieve application → Retrieve documents → Execute AI verification → Save verification results → Return response

get_verifications()

Item	Details
Purpose	Retrieve AI verification records.
Repository	<code>get_verifications()</code>
Validation	Validate pagination, filters, verification status, and application filters.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve verification records → Apply filters & pagination → Return response

get_verification()

Item	Details
Purpose	Retrieve AI verification details.
Repository	<code>get_verification_by_id()</code>
Validation	Validate Verification UUID.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>VerificationNotFoundException</code>
Logic	Validate ID → Retrieve verification → Return response

retry_verification()

Item	Details
Purpose	Re-run AI verification for an application.
Repository	<code>get_verification_by_id()</code> , <code>update_verification()</code>
Validation	Verification record exists. Associated documents must be available.
Permission	Admission Officer, College Admin
Exceptions	<code>VerificationNotFoundException</code> , <code>ValidationException</code>
Logic	Validate request → Retrieve verification → Execute AI verification again → Update verification results → Return success response

update_verification_status()

Item	Details
Purpose	Update the verification status after AI or manual review.
Repository	<code>get_verification_by_id()</code> , <code>update_verification_status()</code>
Validation	Verification exists. Status transition must be valid.
Permission	Admission Officer, College Admin
Exceptions	<code>VerificationNotFoundException</code> , <code>InvalidStatusException</code>
Logic	Validate ID → Retrieve verification → Update verification status → Return success response

4. Module Notes

Database Tables

- `ai_verifications`
- `applications` (Reference Validation)
- `documents` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User
- AI Verification Service

Module 17 – Payments

Module Information

Property	Value
Module Name	Payments
Depends On	Students, Applications
Used By	Admission Processing, Reports, Notifications, Audit Logs
Primary Table	<code>payments</code>

1. Module Overview

Purpose

The **Payments** module manages admission-related fee payments. It records payment transactions, tracks payment status, and provides payment information to dependent modules.

Responsibilities

- Initiate payments.
- Record payment transactions.
- Verify payment status.
- Retrieve payment history.
- Process payment refunds (if applicable).

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/payments</code>	<code>create_payment()</code>	Initiate a payment
GET	<code>/payments</code>	<code>get_payments()</code>	Retrieve all payments
GET	<code>/payments/{id}</code>	<code>get_payment()</code>	Retrieve payment details
PATCH	<code>/payments/{id}/verify</code>	<code>verify_payment()</code>	Verify payment status
PATCH	<code>/payments/{id}/refund</code>	<code>refund_payment()</code>	Process payment refund

(Endpoints are based on your API specification.)

3. Service Implementation

create_payment()

Item	Details
Purpose	Create a payment transaction for an application.
Repository	<code>get_application_by_id()</code> , <code>create_payment()</code>
Validation	Application exists. Payment amount must be valid. Duplicate successful payments should not be allowed.
Permission	Student
Exceptions	<code>ApplicationNotFoundException</code> , <code>PaymentAlreadyExistsException</code> , <code>ValidationException</code>
Logic	Validate request → Verify application → Create payment transaction → Return payment details

get_payments()

Item	Details
Purpose	Retrieve payment records.
Repository	<code>get_payments()</code>
Validation	Validate pagination, filters, sorting, payment status, and date filters.
Permission	Super Admin, College Admin, Student (Own Payments)
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve payments → Apply filters & pagination → Return response

get_payment()

Item	Details
Purpose	Retrieve payment details.
Repository	<code>get_payment_by_id()</code>
Validation	Validate Payment UUID.
Permission	Super Admin, College Admin, Student (Own Payment)
Exceptions	<code>PaymentNotFoundException</code>
Logic	Validate ID → Retrieve payment → Return response

verify_payment()

Item	Details
Purpose	Verify payment completion and update payment status.
Repository	<code>get_payment_by_id()</code> , <code>update_payment_status()</code>
Validation	Payment exists. Payment gateway response must be valid.
Permission	System, College Admin
Exceptions	<code>PaymentNotFoundException</code> , <code>PaymentVerificationException</code>
Logic	Validate payment → Verify gateway response → Update payment status → Return success response

refund_payment()

Item	Details
Purpose	Process a payment refund.
Repository	<code>get_payment_by_id()</code> , <code>update_refund_status()</code>
Validation	Payment exists. Payment must be eligible for refund.
Permission	Super Admin, College Admin
Exceptions	<code>PaymentNotFoundException</code> , <code>RefundNotAllowedException</code>
Logic	Validate payment → Verify refund eligibility → Process refund → Update refund status → Return success response

4. Module Notes

Database Tables

- `payments`
- `applications` (Reference Validation)
- `students` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User
- Payment Gateway Service

Module 18 – Notifications

Module Information

Property	Value
Module Name	Notifications
Depends On	Users, Students
Used By	Applications, Payments, Documents, Admission Cycles
Primary Table	<code>notifications</code>

1. Module Overview

Purpose

The **Notifications** module manages system-generated notifications sent to users during the admission process. It informs users about application updates, document verification, payment status, admission announcements, and other important events.

Responsibilities

- Create notifications.
 - Send notifications.
 - Retrieve notification history.
 - Mark notifications as read.
 - Delete notifications.
-

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/notifications</code>	<code>create_notification()</code>	Create a notification
GET	<code>/notifications</code>	<code>get_notifications()</code>	Retrieve all notifications
GET	<code>/notifications/{id}</code>	<code>get_notification()</code>	Retrieve notification details
PATCH	<code>/notifications/{id}/read</code>	<code>mark_as_read()</code>	Mark notification as read
DELETE	<code>/notifications/{id}</code>	<code>delete_notification()</code>	Soft delete notification

(Endpoints are based on your API specification.)

3. Service Implementation

create_notification()

Item	Details
Purpose	Create and send a notification to a user.
Repository	<code>get_user_by_id()</code> , <code>create_notification()</code>
Validation	Recipient user must exist. Notification title and message are required.
Permission	System, Super Admin, College Admin
Exceptions	<code>UserNotFoundException</code> , <code>ValidationException</code>
Logic	Validate request → Verify recipient → Create notification → Send notification → Return success response

get_notifications()

Item	Details
Purpose	Retrieve notification records.
Repository	<code>get_notifications()</code>
Validation	Validate pagination, filters, sorting, read status, and notification type filters.
Permission	Super Admin, College Admin, Student (Own Notifications)
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve notifications → Apply filters & pagination → Return response

get_notification()

Item	Details
Purpose	Retrieve notification details.
Repository	<code>get_notification_by_id()</code>
Validation	Validate Notification UUID.
Permission	Super Admin, College Admin, Student (Own Notification)
Exceptions	<code>NotificationNotFoundException</code>
Logic	Validate ID → Retrieve notification → Return response

mark_as_read()

Item	Details
Purpose	Mark a notification as read.
Repository	<code>get_notification_by_id()</code> , <code>update_notification_status()</code>
Validation	Notification exists and belongs to the current user.
Permission	Student, Admission Officer, College Admin
Exceptions	<code>NotificationNotFoundException</code> , <code>AccessDeniedException</code>
Logic	Validate ID → Verify ownership → Update read status → Return success response

delete_notification()

Item	Details
Purpose	Soft delete a notification.
Repository	<code>get_notification_by_id()</code> , <code>delete_notification()</code>
Validation	Notification exists.
Permission	Student (Own Notification), Super Admin
Exceptions	<code>NotificationNotFoundException</code>
Logic	Validate ID → Retrieve notification → Soft delete notification → Return success response

4. Module Notes

Database Tables

- `notifications`

- `users` (Reference Validation)
- `students` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User
- Notification Service (Email/SMS/In-App)

Module 19 – Chat

Module Information

Property	Value
Module Name	Chat
Depends On	Users
Used By	Students, Admission Officer, College Admin
Primary Table	<code>chat_history</code>

1. Module Overview

Purpose

The **Chat** module manages communication between students and admission staff. It stores conversations, enables message exchange, and maintains chat history for future reference.

Responsibilities

- Send messages.

- Retrieve chat history.
- View conversations.
- Mark messages as read.
- Maintain communication records.

2. API Specifications

Method	Endpoint	Service Function	Description
POST	<code>/chat/messages</code>	<code>send_message()</code>	Send a message
GET	<code>/chat/conversations</code>	<code>get_conversations()</code>	Retrieve conversations
GET	<code>/chat/messages/{conversation_id}</code>	<code>get_messages()</code>	Retrieve chat history
PATCH	<code>/chat/messages/{id}/read</code>	<code>mark_message_read()</code>	Mark message as read
DELETE	<code>/chat/messages/{id}</code>	<code>delete_message()</code>	Soft delete message

(Endpoints are based on your API specification.)

3. Service Implementation

send_message()

Item	Details
Purpose	Send a message between users.
Repository	<code>get_user_by_id()</code> , <code>create_message()</code>
Validation	Sender and receiver must exist. Message content cannot be empty.
Permission	Student, Admission Officer, College Admin
Exceptions	<code>UserNotFoundException</code> , <code>ValidationException</code>
Logic	Validate request → Verify sender and receiver → Save message → Return success response

get_conversations()

Item	Details
Purpose	Retrieve all conversations of the current user.
Repository	<code>get_conversations()</code>
Validation	Validate pagination and filters.
Permission	Student, Admission Officer, College Admin
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve conversations → Apply filters & pagination → Return response

get_messages()

Item	Details
Purpose	Retrieve chat history for a conversation.
Repository	<code>get_messages()</code>
Validation	Conversation must exist and user must have access.
Permission	Student, Admission Officer, College Admin
Exceptions	<code>ConversationNotFoundException</code> , <code>AccessDeniedException</code>
Logic	Validate request → Verify conversation → Retrieve messages → Return response

mark_message_read()

Item	Details
Purpose	Mark a message as read.
Repository	<code>get_message_by_id()</code> , <code>update_message_status()</code>
Validation	Message exists and belongs to the current user.
Permission	Student, Admission Officer, College Admin
Exceptions	<code>MessageNotFoundException</code> , <code>AccessDeniedException</code>
Logic	Validate ID → Verify ownership → Update read status → Return success response

delete_message()

Item	Details
Purpose	Soft delete a chat message.
Repository	<code>get_message_by_id()</code> , <code>delete_message()</code>

Item	Details
Validation	Message exists.
Permission	Sender, Super Admin
Exceptions	<code>MessageNotFoundException</code>
Logic	Validate ID → Retrieve message → Soft delete message → Return success response

4. Module Notes

Database Tables

- `chat_history`
- `users` (Reference Validation)

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`
- `common/validators.py`

Dependencies

- Database Session
- Current User
- WebSocket / Real-time Messaging Service (if enabled)

Module 20 – Audit Logs

Module Information

Property	Value
Module Name	Audit Logs
Depends On	Users
Used By	All Modules
Primary Table	<code>audit_logs</code>

1. Module Overview

Purpose

The **Audit Logs** module records all important system activities performed by users and system processes. It provides traceability, accountability, and supports system monitoring and troubleshooting.

Responsibilities

- Record user activities.
 - Store system events.
 - Retrieve audit logs.
 - Filter audit history.
 - Support monitoring and compliance.
-

2. API Specifications

Method	Endpoint	Service Function	Description
POST	/audit-logs	create_audit_log()	Record a system activity
GET	/audit-logs	get_audit_logs()	Retrieve audit logs
GET	/audit-logs/{id}	get_audit_log()	Retrieve audit log details

(Endpoints are based on your API specification.)

3. Service Implementation

create_audit_log()

Item	Details
Purpose	Record a user or system activity.
Repository	create_audit_log()
Validation	User ID (if applicable), module name, action, and timestamp are required.
Permission	System (Internal)

Item	Details
Exceptions	<code>ValidationException</code>
Logic	Collect activity details → Create audit log → Save record → Return success response

get_audit_logs()

Item	Details
Purpose	Retrieve audit log records.
Repository	<code>get_audit_logs()</code>
Validation	Validate pagination, filters, sorting, user, module, action, and date filters.
Permission	Super Admin
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve audit logs → Apply filters & pagination → Return response

get_audit_log()

Item	Details
Purpose	Retrieve audit log details.
Repository	<code>get_audit_log_by_id()</code>
Validation	Validate Audit Log UUID.
Permission	Super Admin
Exceptions	<code>AuditLogNotFoundException</code>
Logic	Validate ID → Retrieve audit log → Return response

4. Module Notes

Database Tables

- `audit_logs`
- `users` (Reference Validation)

Common Components

- `common/exceptions.py`

- `common/helpers.py`
- `common/response.py`

Dependencies

- Database Session
- Current User

Module 21 – Reports

Module Information

Property	Value
Module Name	Reports
Depends On	Applications, Students, Payments, Admission Cycles
Used By	Dashboard, College Admin, Super Admin
Primary Table	<i>(No Primary Table – Uses Multiple Modules)</i>

1. Module Overview

Purpose

The **Reports** module generates admission-related reports by collecting data from multiple modules. It provides analytical and downloadable reports for administrators and management.

Responsibilities

- Generate admission reports.
 - Generate student reports.
 - Generate payment reports.
 - Export reports.
 - Provide report summaries.
-

2. API Specifications

Method	Endpoint	Service Function	Description
GET	/reports/admissions	get_admission_report()	Generate admission report
GET	/reports/students	get_student_report()	Generate student report
GET	/reports/payments	get_payment_report()	Generate payment report
GET	/reports/dashboard	get_dashboard_report()	Generate dashboard summary
GET	/reports/export	export_report()	Export report (PDF/Excel/CSV)

(Endpoints are based on your API specification.)

3. Service Implementation

get_admission_report()

Item	Details
Purpose	Generate admission statistics and application summaries.
Repository	get_admission_report()
Validation	Validate admission cycle, institution, department, and date filters.
Permission	Super Admin, College Admin
Exceptions	ValidationException
Logic	Validate request → Retrieve application data → Generate report → Return response

get_student_report()

Item	Details
Purpose	Generate student-related reports.
Repository	get_student_report()
Validation	Validate course, department, institution, and status filters.
Permission	Super Admin, College Admin
Exceptions	ValidationException
Logic	Validate filters → Retrieve student data → Generate report → Return response

get_payment_report()

Item	Details
Purpose	Generate payment collection reports.
Repository	<code>get_payment_report()</code>
Validation	Validate payment status, payment mode, and date filters.
Permission	Super Admin, College Admin
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve payment data → Generate report → Return response

get_dashboard_report()

Item	Details
Purpose	Generate summary statistics for the dashboard.
Repository	<code>get_dashboard_report()</code>
Validation	Validate institution and admission cycle filters.
Permission	Super Admin, College Admin
Exceptions	<code>ValidationException</code>
Logic	Retrieve dashboard statistics → Prepare summary → Return response

export_report()

Item	Details
Purpose	Export reports in supported formats.
Repository	<code>get_report_data()</code>
Validation	Validate report type and export format (PDF, Excel, CSV).
Permission	Super Admin, College Admin
Exceptions	<code>InvalidReportFormatException</code> , <code>ValidationException</code>
Logic	Validate request → Retrieve report data → Generate export file → Return file response

4. Module Notes

Database Tables

- `applications`

- `students`
- `payments`
- `admission_cycles`

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`

Dependencies

- Database Session
- Current User

Module 22 – Dashboard

Module Information

Property	Value
Module Name	Dashboard
Depends On	Applications, Students, Payments, Documents, Admission Cycles
Used By	Super Admin, College Admin, Admission Officer
Primary Table	<i>(No Primary Table – Uses Multiple Modules)</i>

1. Module Overview

Purpose

The **Dashboard** module provides a consolidated view of admission statistics and system activities. It collects data from multiple modules and displays key metrics for administrators.

Responsibilities

- Display admission statistics.
- Display application status summary.

- Display payment statistics.
- Display document verification status.
- Provide dashboard analytics.

2. API Specifications

Method	Endpoint	Service Function	Description
GET	<code>/dashboard/summary</code>	<code>get_dashboard_summary()</code>	Retrieve dashboard summary
GET	<code>/dashboard/applications</code>	<code>get_application_statistics()</code>	Retrieve application statistics
GET	<code>/dashboard/payments</code>	<code>get_payment_statistics()</code>	Retrieve payment statistics
GET	<code>/dashboard/documents</code>	<code>get_document_statistics()</code>	Retrieve document statistics
GET	<code>/dashboard/recent-activities</code>	<code>get_recent_activities()</code>	Retrieve recent system activities

(Endpoints are based on your API specification.)

3. Service Implementation

get_dashboard_summary()

Item	Details
Purpose	Retrieve overall dashboard statistics.
Repository	<code>get_dashboard_summary()</code>
Validation	Validate institution and admission cycle filters.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>ValidationException</code>
Logic	Validate request → Collect summary data → Return dashboard response

get_application_statistics()

Item	Details
Purpose	Retrieve application-related statistics.
Repository	<code>get_application_statistics()</code>
Validation	Validate admission cycle, course, and department filters.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>ValidationException</code>
Logic	Validate filters → Retrieve application statistics → Return response

get_payment_statistics()

Item	Details
Purpose	Retrieve payment statistics.
Repository	<code>get_payment_statistics()</code>
Validation	Validate payment status and date filters.
Permission	Super Admin, College Admin
Exceptions	<code>ValidationException</code>
Logic	Validate filters → Retrieve payment statistics → Return response

get_document_statistics()

Item	Details
Purpose	Retrieve document upload and verification statistics.
Repository	<code>get_document_statistics()</code>
Validation	Validate admission cycle and verification status filters.
Permission	Super Admin, College Admin, Admission Officer
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve document statistics → Return response

get_recent_activities()

Item	Details
Purpose	Retrieve recent admission-related activities.
Repository	<code>get_recent_activities()</code>
Validation	Validate activity filters and pagination.

Item	Details
Permission	Super Admin, College Admin
Exceptions	<code>ValidationException</code>
Logic	Validate request → Retrieve recent activities → Apply pagination → Return response

4. Module Notes

Database Tables

- `applications`
- `students`
- `payments`
- `documents`
- `audit_logs`

Common Components

- `common/exceptions.py`
- `common/helpers.py`
- `common/response.py`

Dependencies

- Database Session
- Current User